Apunte 09 - Iteradores y Colecciones en Java

Introducción

A continuación vamos a comenzar el proceso de analizar las alternativas de implementación de estructuras de datos en java al nivel que requerimos en Backend de Aplicaciones para comprender el funcionamiento de las colecciones nativas de Java y poder aprovechar su potencia pero sin profundizar en los conceptos reales de la programación de estructuras de datos no nativas que sería material para otra asignatura completa.

En pocas palabras, el objetivo de programar una estructura de datos es independizar el concepto que muestra la interfaz de los objetos de dicha estructura del almacenamiento subyacente y real de los datos. Es decir que vamos a encapsular dentro de una clase un esquema de almacenamiento de datos de forma tal que el usuario de las instancias de la clase no se tenga que preocupar por dicho almacenamiento pero que pueda utilizar el objeto como si fuera el concepto que implementa.

Para esto vamos a implementar una lista sobre un vector o array en Java, como ya hemos discutido cuando analizamos arrays o vectores estos tiene ciertas particularidades que los hacen hasta cierto punto estáticos ya que una vez creados no pueden cambiar su tamaño y agregan la particularidad de requerir todos los elementos del mismo tipo. Una lista en cambio es una estructura de datos que agrega elementos a medida que se requiere y no tiene un tamaño fijo, mantiene el concepto de orden de inserción y el de estructura lineal, es decir que un elemento le sigue a otro y cada elemento tiene 0 o solo 1 elemento siguiente o anterior.

La estructura de datos natural para implementar este concepto son las listas vinculadas, pero no corresponde a esta asignatura el análisis del desarrollo de las listas vinculadas (fuera que luego las usemos en Java), por lo que vamos a implementar el concepto de la lista pero utilizando como repositorio de datos interno los vectores que ya conocemos.

Lista sobre vectores o Arrays

Vamos entonces con la clase que mencionamos:



En teste caso el problema fundamental radica en ocultar las desventajas de los vectores al usuario de la lista, es decir el usuario de la lista debe poder utilizar la misma como si fuera una lista de Python o Javascript que no tiene limitación en cuanto a la cantidad, y un detalle extra que vamos a notar es que deberemos trabajar en todos los casos en base a Object para lograr que nuestra ListaArray pueda contener objetos del tipo que haga falta.

En primer lugar analicemos atributos:

```java
public class ListaArrayBasica {
    // el arreglo que contendrá los elementos...
    private Object[] elementos;

    // el tamaño inicial del arreglo...
    private int capacidadInicial;

    // la cantidad de casillas realmente usadas...
    private int count;

    // índice de la posición actual del iterador
    private int actual;

```

El prime atributo que vemos es el vector de elementos donde vamos a almacenar los objetos que sean agregados a la lista. Este vector es lo que hemos denominado almacenamiento subyacente y el usuario no tendrá contacto con él.

El segundo parámetro que vemos es la capacidad inicial de dicho vector y nos servirá para volver al estado inicial la lista usando el método `clear()`

El siguiente es el parámetro que indica la cantidad de posiciones del vector utilizadas actualmente y aquí vamos a aclarar una situación. Evidentemente no podemos crear el vector vacío y agrandarlo cada vez que el usuario desee agregar un nuevo elemento porque eso provocaría un importante overhead de trabajo al copiar cada vez que recreamos el vector los elementos del vector viejo al vector nuevo, esto se debe a que los vectores en java no permiten de manera natural redimensionar su contenido. Retomaremos este tema más adelante.

Definimos 2 constructores:

```
22
23      /** Crea una lista con capacidad inicial de 10 casilleros, pero ninguno ...*/
        new *
27      public ListaArrayBasica() {
28
29          this( capacidad: 10);
30      }
31
32      /** Crea una lista con capacidadInicial casilleros de capacidad, pero ninguno ...*/
        new *
39      public ListaArrayBasica(int capacidad) {
40          if (capacidad <= 0) {
41              capacidad = 10;
42          }
43          elementos = new Object[capacidad];
44          capacidadInicial = capacidad;
45          count = 0;
46      }
```

El primero no recibe parámetros y el segundo recibe la capacidad inicial, este último está orientado a que si sabemos que vamos a manejar gran cantidad de datos, creemos la lista ya con la cantidad aproximada para evitar o reducir la cantidad de veces que hará falta redimensionar el vector subyacente.

Notar también que ambos constructores además del vector y la capacidad inicial, establecen el valor de count en cero lo que indica que la lista inicia vacía.

Ahora bien vamos con el comportamiento fundamental que implica agregar un elemento, este comportamiento es homólogo al append de Python que tantas veces se usa en AED.

```
47
48      /** Añade al final de la lista el objeto e. La inserción será rechazada si la ...*/
        new *
55      public boolean agregar(Object e) {
56          if (e == null) return false;
57
58          if (count == elementos.length) this.asegurarCapacidad( capacidadMinima: elementos.length * 2);
59
60          elementos[count] = e;
61          count++;
62          return true;
63      }
64
```

El método valida que el objeto a agregar no sea null y luego, si el mundo fuera ideal y los vectores infinitos, se limita a agregar el elemento en la posición que indique count (cómo el primer subíndice siempre será 0, la cantidad siempre indicará el próximo subíndice a ser utilizado, esta realidad merece una revisión por parte del alumno en caso de que no esté clara). Eso producen las líneas 60 y 61.

Pero, como el mundo nunca es ideal y los vectores en Java no son infinitos, la línea 58 es la que determina si en el vector subyacente aún hay espacio para agregar el elemento o no, cuando la cantidad actual de elementos del vector es igual al tamaño del vector subyacente quiere decir que el vector subyacente está completamente ocupado y por lo tanto que no hay espacio para agregar un nuevo elemento. En este caso será necesario redimensionar el vector y esa es la función que cumple el método asegurarCapacidad:

```java
126     private void asegurarCapacidad(int capacidadMinima) {
127         if (capacidadMinima == elementos.length) return;
128         if (capacidadMinima < count) return;
129 
130         Object[] temp = new Object[capacidadMinima];
131         System.arraycopy(elementos, srcPos: 0, temp, destPos: 0, count);
132         elementos = temp;
133     }
```

Este método, luego de algunas validaciones, crea un nuevo vector más grande que el anterior, notar que no agrega una sola posición sino que agrega una cantidad de elementos libres para no tener que volver a ejecutar este método ante cada nuevo elemento atravesado el umbral de capacidad inicial.

Una vez creado el vector la sentencia `System.arraycopy(elementos, 0, temp, 0, count);` copia el contenido del vector actual al primer segmento del nuevo vector quedando así nuevos elementos no utilizados. Finalmente se reemplaza el vector subyacente con el nuevo vector y al volver al método agregar este entiendo que el vector tine elementos libres y agrega el nuevo elemento como si nada hubiera pasado.

A partir de aquí la clase implementa varios métodos para dar soporte a distintas acciones en la lista y las herramientas al usuario para poder usarla, no vamos a describir cada uno de los métodos queda para el alumno revisar estos y consultar en clase si queda alguna duda al respecto.

Probando la Lista sobre Array básica

En la clase `PruebaListaArray` hay bastante código probando la Lista descrita en el punto anterior donde se puede observar el uso de los comportamientos que hemos mencionado y también el del resto de los comportamientos desarrollados en la clase.

Ahora bien, en muchos problemas será necesario tomar una lista y recorrerla para obtener algún resultado relevante. Por ejemplo, si una aplicación crea una lista de cuentas de Inversion se podría esperar que en algún momento se nos pida el saldo promedio entre todas las cuentas que están en la lista. El método que recorra la lista, tome y acumule el saldo de cada cuenta y finalmente retorne el promedio, no debería formar parte de la clase `ListaArray`, tal método estaría suponiendo que la lista tiene objetos de la clase Inversion y eso sería falso en general, la clase ListaArray fue planteada para suponer elementos polimóficos. Ese método sólo sería utilizable si efectivamente se crea una instancia de la clase `ListaArray` en la que se inserten objetos que representen Cuentas de Inversion.

Si bien como sabemos que en esta implementación existe un vector subyacente la opción de hacer el recorrido con un `for` usando el método `size()` que retorna la cantidad y el método `get(i)` que devuelve el elemento en la posición i podríamos lograr un recorrido de la lista desde afuera del objeto. En este ejemplo de la clase `PruebaListaArray`, recorremos una lista en la que hemos agregado cadenas con nombres:

```
61          System.out.println("Contenido usando get: ");
62          for (int i = 0; i < la.size(); i++)
63          {
64              String x = (String) la.get(i);
65              System.out.println(x);
66          }
```

sin embargo, nos estamos basando en que conocemos la estructura de almacenamiento y que también conocemos que soporta acceso directo con lo que usar get(i) no supone un overhead de trabajo, cosa que no pasa con otras estructuras de datos de soporte subyacente. Por lo tanto será necesario encontrar un esquema que permita recorrer el contenido de la lista sin tener ningún conocimiento de su estructura o almacenamiento interno y esa situación se resuelve utilizando el patrón iterador.

En una primera versión absolutamente simplificada y trivial del patrón pensemos en cuáles son los comportamientos obligatorios que requeriríamos al intentar agregarla a nuestra lista la capacidad de iteración:

- Un comportamiento para reiniciar el iterador que permita resetear este al primer elemento de la lista.
- Un comportamiento que permita saber si hay más elementos de forma de poder saber si debemos seguir recorriendo o debemos detener el recorrido porque se terminó el contenido de la lista.
- Un comportamiento que permita obtener el elemento actual del recorrido.
- Un comportamiento que permita avanzar al elemento siguiente en el recorrido.

Si nuestra clase soporta esos comportamiento podríamos pensar en un recorrido de la lista de esta forma:

```
68          System.out.println("Contenido usando iterador: ");
69          la.iniciarIterador();
70          while(la.hayMas())
71          {
72              String x = (String) la.getActual();
73              System.out.println(x);
74              la.siguiente();
75          }
```

notar que en este caso no tenemos conocimiento de la estructura interna de la lista, de hecho podría estar implementada incluso sobre una estructura no lineal y de todas maneras nosotros podemos recorrer todos sus elementos de forma lineal sin preocuparnos por su estructura interna.

Primera versión trivial del iterador

```java
    /**
     * Metodo que inicia la posibilidad de iteración a partir del primer elemento del vector
     */
    public void iniciarIterador() {
        actual = 0;
    }

    public boolean hayMas() {
        if(isEmpty()) { return false; }
        if(actual >= size() - 1) { return false; }
        return true;
    }

    public Object getActual() {
        if (actual >= count) {
            throw new NoSuchElementException("next(): no quedan elementos por recorrer...");
        }
        return elementos[actual];
    }

    public void siguiente() {
        if (!hayMas()) {
            throw new NoSuchElementException("next(): no quedan elementos por recorrer...");
        }
        actual++;
    }

```

Como vemos no hay mayor complejidad en la implementación del iterador directamente escrito como métodos de la clase, cabe aclarar que estamos usando aquí el último de los atributos que no explicamos al principio, el atributo actual contiene el índice del elemento actual en la iteración.

Por ello el método `iniciarIterador` pone el valor 0 en el atributo actual para que apunte al primer elemento de la lista. Luego el método `getActual()` retorna el elemento que está en la posición indicada por el atributo actual. Los métodos `hayMas()` y `siguiente()` también se apoyan en dicho atributo para determinar si hay más contenido por recorrer y avanzar al siguiente elemento.

Esta podría ser una solución pero tiene un problema particular y es que si lo implementamos de esta forma solo podemos plantear un recorrido por vez, ya que si estamos en medio de un recorrido y queremos volver a iniciar por ejemplo para realizar una búsqueda implicaría modificar el recorrido actual y ese problema hace que la solución no sea viable.

¿Cómo resolvemos esto entonces?, la respuesta a esta pregunta es ***otro*** objeto capada de llevar a cabo la iteración de la lista, pero, si programamos otra clase para que itere la lista esta nueva clase tendría que conocer los detalles internos de la clase `ListaArray` y eso rompería el encapsulamiento, la solución a esta situación está dada, en Java, por las clases internas.

Clases internas en Java

Una clase interna es una clase que se define dentro del ámbito de otra clase, como un miembro más. Así, una clase que contenga una o más clases internas tendrá miembros que serán o bien atributos propios, o bien métodos propios, o bien clases internas. No hay demasiada novedad en una clase interna, de hecho es en todo similar a una clase común, sin embrago, bastará con saber que si una clase B se declara como interna de otra clase A, entonces los métodos de la clase B pueden acceder en forma directa a todos los atributos y métodos de A (sin importar si se declararon private o no). Esto tiene sentido si se considera que la clase B es

miembro de A, y por lo tanto tiene acceso a los otros miembros de A (de la misma forma que los métodos de A tienen acceso a los atributos de la misma clase A).

Por lo tanto, y aplicando lo dicho, en el modelo la clase `ListaArrayMejorada` declara como clase interna a la clase `IteradorLineal` de acuerdo al siguiente esquema:

```
236     public class IteradorLineal {
237
238         // índice de la posición actual del iterador
239         private int actual;
240         public IteradorLineal() {
241
242             actual = 0;
243         }
244
245         public boolean hayMas() {
246             if(isEmpty()) { return false; }
247             if(actual >= size() - 1) { return false; }
248             return true;
249         }
250
251         public Object getActual() {
252             if (actual >= count) {
253                 throw new NoSuchElementException("next(): no quedan elementos por recorrer...");
254             }
255             return elementos[actual];
256         }
257
258         public void siguiente() {
259             if (!hayMas()) {
260                 throw new NoSuchElementException("next(): no quedan elementos por recorrer...");
261             }
262             actual++;
263         }
264
265     }
```

Ahora tenemos el iterador de forma tal que permite generar tantos objetos iteradores independientes como haga falta y que además pueden acceder a los atributos de la clase `ListaArray` sin romper el encapsulamiento. el problemas es que vamos a necesitar también agregar en la clase lista la capacidad de obtener un iterador de esta para poder utilizarlo, esa función la cumple el método:

```
227     public IteradorLineal iterador() {
228         return new IteradorLineal();
229     }
230
```

Que devuelve una instancia de `IteradorLineal` para la clase `ListaArray` y a partir del cual podemos realizar el mismo recorrido que hicimos con el iterador básico con mínimas diferencias, el recorrido quedará como sigue:

```
        System.out.println("Contenido usando iterador: ");
        ListaArrayMejorada.IteradorLineal it = la.iterador();
        while(it.hayMas())
        {
            String x = (String) it.getActual();
            System.out.println(x);
            it.siguiente();
        }

```

La diferencia principal radica en que lo que antes hacíamos con la referencia a la clase `ListaArray` ahora lo hacemos con la referencia de tipo `IeradorLienal` pero el concepto de mantiene de la misma forma.

Clases anónimas en java - clases inline

En muchos casos en Java las clases sólo se la implementa para disponer del método que permita realizar cierta tarea necesaria y obligada por una interfaz o una clase abstracta.

Estas clases que se implementan con este objetivo único en muchos casos sólo serán usadas una única vez, para crear un único objeto. Se trata de una clase "de un solo uso", y sin embargo al declararla se le coloca un nombre y se obliga con ello a la máquina virtual no sólo a cargarla en memoria cuando el programa arranca, sino también a registrarla y darle soporte en tiempo de ejecución, por si en otro momento surge otro new para crear nuevas instancias. En casos así, no parece siquiera necesario que la clase lleve un nombre y sobrecargar de trabajo a la máquina virtual. Se puede recurrir entonces a una clase anónima.

Una clase anónima es una clase interna sin nombre, que se crea, se compila y se carga en memoria en el momento que se necesita, con el objetivo de contener, por lo general, un único método simple o lo conjunto acotado de métodos sencillos. El compilador genera un .class para la clase anónima con un nombre compuesto por el nombre de la clase outer, el signo $, y un número de orden que será correlativo por orden de aparición de cada clase anónima.

La restricción para crear una clase anónima es la existencia de una clase abstracta de la que nuestra clase anónima va a derivar o de una interfaz que nuestra clase anónima va a implementar y la forma de crearla es utilizando dicha clase Abstracta o Interfaz como si fuera concreta pero agregando inmediatamente las llaves y la declaración del cuerpo de la clase, por esto también se las denomina clases inline.

Ejemplo de Uso

Supongamos que tienes la siguiente interfaz:

```
interface Procesador {
    void procesar();
}
```

Si deseas crear una instancia única de esta interfaz y definir su método procesar(), puedes usar una clase anónima:

```
...
Procesador miProcesador = new Procesador() {
    @Override
    public void procesar() {
        System.out.println("Procesamiento en clase anónima.");
    }
};
...
```

En resumen, las clases anónimas son una herramienta valiosa para crear instancias de clases que implementan interfaces o extienden clases abstractas en el lugar donde se necesitan. Son particularmente útiles para implementaciones temporales y reducen la complejidad del código al evitar la creación de clases separadas para casos simples. Sin embargo, cuando se trata de implementaciones más largas o reutilizables, es posible que sea más apropiado usar clases internas o las expresiones lambda que veremos más adelante.

Clases Parametrizadas en Java - Generics

En nuestra última versión de la clase ArrayList, tenemos la situación que todos los datos almacenados en la lista son de tipo Object y por lo tanto si bien puedo almacenar cualquier cosa necesito hacer casting luego para poder usar los objetos no tengo forma de controlar que se agreguen objetos de un solo tipo a la lista. Si bien podríamos programar un control usando el método getClass() y controlando si el objeto que se inserta es de la misma clase que el que figura primero en la lista, y de ser así, la inserción se acepta. Incorporando esa idea en todo método de inserción o en todo método que necesite comparar un objeto tomado como parámetro con el contenido de la lista, aseguramos la homogeneidad. Sin embargo, esto no resuelve el casting necesario a los objetos obtenidos de la lista.

A partir de la versión 1.5 del lenguaje Java o simplemente de Java 5 se incorpora un mecanismo de parametrización de clases con un tipo específico, designado como Generics y que aquí designaremos como clases parametrizadas. A partir de Java 1.5, las colecciones o estructuras de datos predefinidas del lenguaje (como LinkedList, ArrayList o HashMap, por ejemplo) pueden definirse en forma clásica, o pueden definirse también con un tipo como parámetro, donde podrán insertarse en la colección objetos de esti tipo:

```
ArrayList a = new ArrayList();
ArrayList<String> b = new ArrayList<>();
```

> Nota: en la primera declaración y creación creamos una lista sobre un arreglo que permite cualquier tipo de objetos, en la segunda, en cambio, creamos la misma lista sobre un arreglo pero de forma que solo podemos operar con Objetos de tipo String.

En las definiciones anteriores, la lista a admite que se inserten en ella objetos de cualquier clase, en forma heterogénea (y esto es lo único que se podía hacer hasta la versión 1.5):

```
a.add("casa");
a.add(new Integer(5));
```

```
        a.add(new Cuenta());
```

Pero la lista b sólo admitirá inserción de objetos de la clase String: cualquier intento de invocar a add() para agregar en b un objeto de otra clase, no compilará:

```
        b.add("casa"); // ok...
        b.add(new Integer(5)); // no compila…
        b.add(new Cuenta()); // no compila…
        b.add("perro"); // ok...
```

Se dice que la clase ArrayList admite otra clase como parámetro, o que está parametrizada. Y el mecanismo que permite hacer eso es el que se designa como generics desde Java 5 en adelante. Todas las clases del paquete java.util que representan colecciones o estructuras de datos, están parametrizadas de la misma forma. Note que es el propio compilador el que se encarga de rechazar una inserción heterogénea si la instancia se creó parametrizada, con lo cual la forma de controlar la homogeneidad es más clara que la que usamos en nuestra implementación. En nuestro caso, el control se hace en tiempo de ejecución y no hay forma de declarar una variable (de tipo SimpleList, por ejemplo) de forma que quede claro en la propia declaración cuál es la clase de objeto que queremos poder añadir a la lista, debemos esperar a la primera inserción para que de allí en adelante el resto de las inserciones se ajusten al primer objeto.

El mecanismo de parametrización de clases está disponible también para las clases del programador. El núcleo del lenguaje Java ha sido modificado a partir de Java 5 para que los programadores puedan incorporar generics en sus propias clases. Por lo tanto, podemos intentar modificar nuestro planteo de `ListaArray`, para que pueda parametrizarse y dejar en manos del compilador el control de tipo de los objetos que agregamos a la lista.

El primer cambio que vamos a notar es en la declaración de la clase puesto que debemos declara la definición del parámetro de tipo. A continuación veamos nuestra `ListaArrayMejorada` pero ahora incluyendo generics para validación del tipo:

```
9   public class ListaArrayMejorada<E>{
10      // el arreglo que contendrá los elementos...
11      private Object[] elementos;
12
13      // el tamaño inicial del arreglo...
14      private final int capacidadInicial;
15
16      // la cantidad de casillas realmente usadas...
17      private int count;
18
```

como podemos observar estamos agregando la letra `E` entre `< >` para terminar el nombre del parámetro que determinará luego el tipo para el resto de código de la clase.

Un elemento particular es que no estamos modificando el tipo del vector y esto tiene que ver con que Java no permite la creación ni manipulación de vectores raw de tipos parametrizados por lo que vamos a mantener el tipo del vector como Object y vamos llevar los cast a los accesos a los elementos de dicho vector.

Luego cada vez que hacemos un método público que va a requerir un objeto desde fuera de la clase vamos a usar `E` para hacer referencia al tipo:

```
43
44  >   /** Añade al final de la lista el objeto e. La inserción será rechazada si la ...*/
51  >   public boolean agregar(E e) {...}
60
61  >   /** Añade el objeto e en la posisicón index de la lista . La inserción será ...*/
72  >   public void insertar(int index, E e) {...}
86
87  >   /** Retorna el objeto contenido en la casilla index. Si el valor de index no ...*/
96  >   public E get(int index) {...}
102
103 >   /** Remueve de la lista el elemento contenido en la posición index. Los ...*/
114 >   public E quitar(int index) {...}
129
130 >   /** Reemplaza el objeto en la posición index por el referido por element, y ...*/
141 >   public E set(int index, E element) {...}
149
```

Como se puede observar cada vez que se hace referencia al tipo se utiliza el parámetro `E`, en este sentido no hay demasiado trabajo extra que hacer en el caso del los métodos que agregan pero cuando en cambio queremos obtener un elemento tenemos la situación en la que deberemos hacer casting para avisarle a Java que estamos seguros de la conversión de tipo, por ejemplo en el caso del método `get()`:

```
87  >   /** Retorna el objeto contenido en la casilla index. Si el valor de index no ...*/
96      public E get(int index) {
97          if (index < 0 || index >= count) {
98              throw new IndexOutOfBoundsException("get(): índice fuera de rango...");
99          }
100         return (E) elementos[index];
101     }
```

Así planteada, la clase ListaArrayMejorada sigue siendo genérica y controla homogeneidad, pero surge un nuevo problema, al definir los elementos del vector de tipo Object y en esencia el compilador entendrá cualquier cosa de tipo `E` como si fuera Object generamos que cualquier acción que necesitemos llevar a cabo sobre estos elementos se reduciran a los métodos de la clase Object. Como vimos en el apartado de Polimorfismo, el tipo de la referencia define la interfaz de la instancia.

Entonces, supongamos que quisiéramos poder comparar los elementos del vector para ordenarlos de mayor a menor primero deberíamos cambiar el vector pero luego además deberíamos indicar que `E` ya no puede ser cualquier object sino cualquier objecto que implemente la interfaz comparable, para ello, al declarar el parámetro de clase E, indicar que esa clase deriva de (o implementa a) Comparable:

```
public class ListaArray<E extends Comparable> {
    ...
}
```

Un detalle a ver es que usamos la palabra extends más allá de que Comparable es una clase, bueno pues, dentro del operador rombo < > la palabra extends representa una restricción y no un hecho del compilador.

Existen muchas particularidades más sobre este tema y su estudio requiere revisar varios aspectos específicos algunos de los cuales difieren de una versión a otra, pero como hemos dicho no estamos en un curso de Java así que dejamos esa investigación para el alumno y nos limitamos a revisar los elementos requeridos.

Agregaremos la definición de los métodos a medida que sean necesarios si lo son.

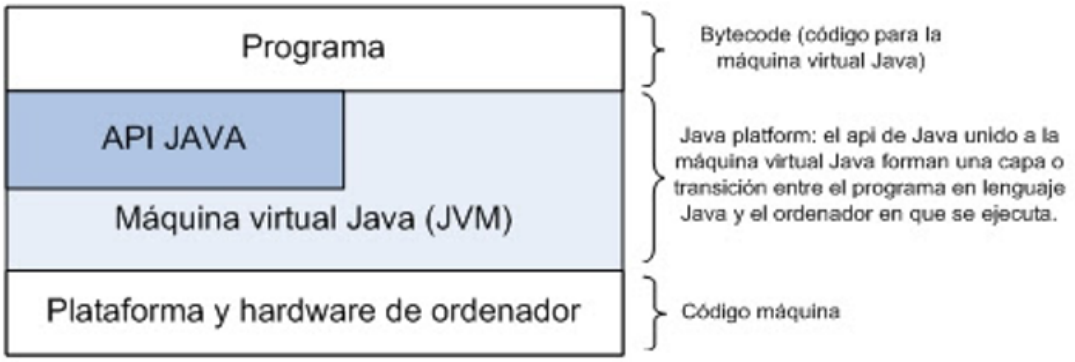
Framework de Colecciones de Java

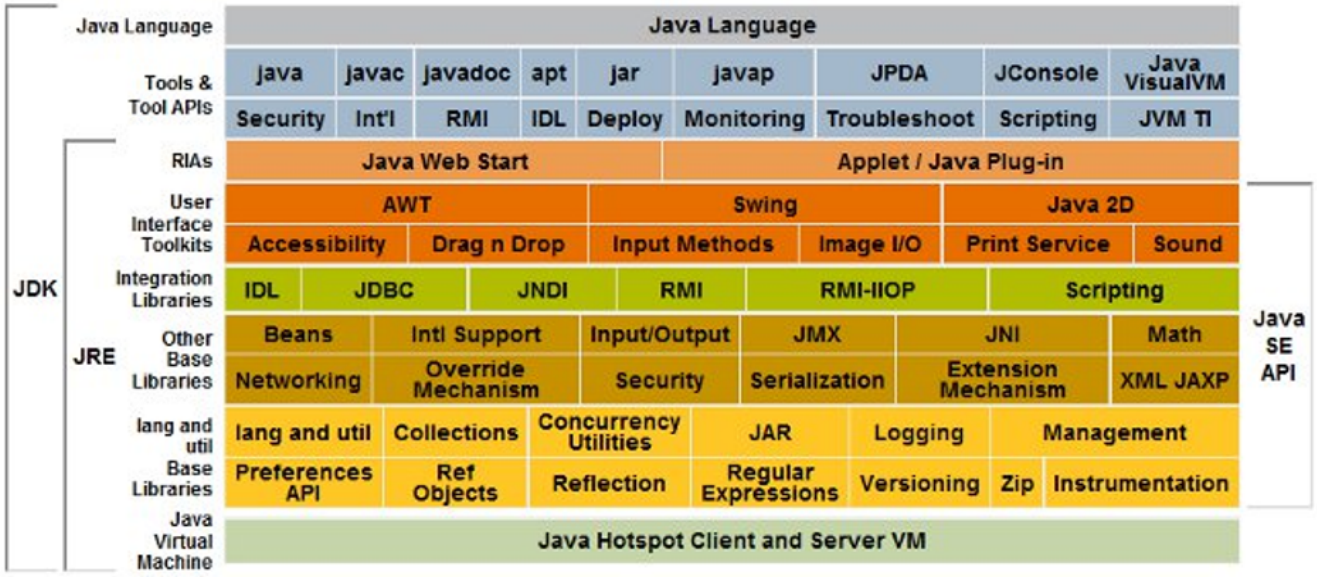
Una breve referencia al conjunto de Apis de Java

La API Java es una interfaz de programación de aplicaciones (API, por sus siglas del inglés: Application Programming Interface) provista por los creadores del lenguaje de programación Java, que da a los programadores los medios para desarrollar aplicaciones Java.

Como el lenguaje Java es un lenguaje orientado a objetos, la API de Java provee de un conjunto de clases utilitarias para efectuar toda clase de tareas necesarias dentro de un programa. La API Java está organizada en paquetes lógicos, donde cada paquete contiene un conjunto de clases relacionadas semánticamente.

Los siguientes esquemas, parte de la documentación de Java, nos dan una idea de cómo funciona el sistema Java y de qué se instala cuando instalamos Java (el paquete JDK) en nuestro ordenador.





La manera que java utiliza para organizar el mundo de clases que implementan las APIs de los esquemas anteriores son las librerías y los paquetes.

Paquetes de clases

Tanto las clases nativas del lenguaje Java como las que pueda definir un programador en un proyecto propio pueden agruparse de acuerdo a objetivos y funcionalidades comunes, y formar colecciones de clases que en Java se llaman paquetes (o packages). Un package es lo que comúnmente se designaría como una librería de clases en otros lenguajes o en otros contextos. En principio, el único criterio para decidir si dos clases deben ir juntas en un package es la funcionalidad común: si ambas clases sirven para tareas y objetivos similares, es común pensar en agruparlas en el mismo package junto a otras clases que también tengan ese objetivo. Así, por ejemplo, todas las clases que originalmente se pensaron para el diseño de interfaces visuales de usuario, se agruparon en el package java.awt y todas las clases nativas pensadas para permitir entrada o salida hacia dispositivos externos se agruparon en el package java.io.

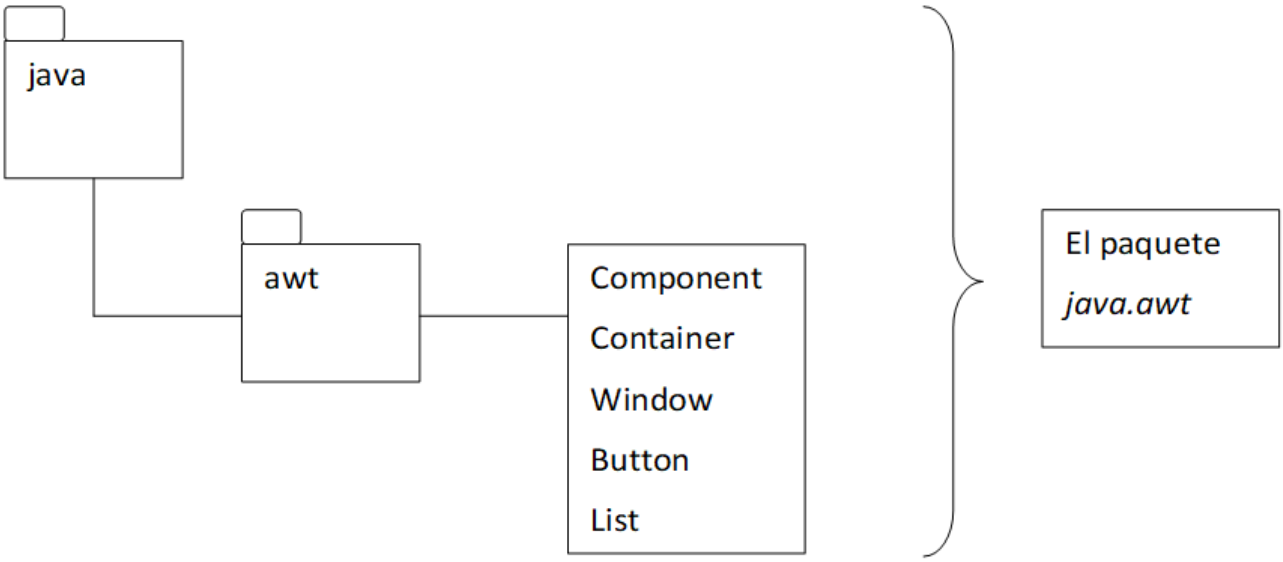
No es obligatorio que las clases de un package tengan entre ellas relación de herencia. De hecho, podrían no tener ninguna relación entre ellas, más que la nombrada de la funcionalidad común. Por otra parte, tampoco hay nada que prohíba que una clase de un package derive de otra ubicada en el mismo package o en otro package diferente. Sólo deberán considerarse los alcances del calificador protected cuando eso ocurra, pues en caso de clases ubicadas en distintos packages la visibilidad de miembros marcados protected será diferente. Remarcamos: un package es una colección de clases que normalmente tienen funcionalidad común, y se agrupan para lograr ciertas ventajas de organización y transparencia en el uso de nombres, que no se tendrían si todas las clases se mantuvieran separadas.

Al hacer que las clases de un proyecto se agrupen en paquetes, el proyecto se organiza y administra mejor. Las clases orientadas a la gestión de la interfaz de usuario pueden ir juntas en un package, mientras que aquellas que representan excepciones propias podrían ir en otro. Las clases específicas del modelo o del problema pueden agruparse en un tercer package y aquellas orientadas a la gestión de entrada o salida hacia archivos o bases de datos pueden ir en un cuarto package. Resultará más sencillo rehusar y distribuir esas clases, documentarlas y mantenerlas.

Por otra parte, el uso de packages permite solucionar potenciales problemas en cuanto a nombres repetidos: suponga que se está desarrollando un proyecto para gestión académica, y se está usando una clase propia

llamada Materia. Si luego se incorpora al proyecto el desarrollo de otro equipo de programadores, y en ese proyecto también hay una clase Materia (o varias clases con el mismo nombre que las del proyecto propio), no habría forma de hacer que el compilador pueda distinguir esas clases a menos que se las haga pertenecer a paquetes diferentes. Un package actúa como un espacio de nombres: el nombre o identificador del package al cual pertenece una clase forma parte del nombre de la clase del mismo modo que la ruta de directorios o carpetas en la cual está grabado un archivo forma parte del nombre físico de ese archivo. Así como no puede haber dos archivos con el mismo nombre en la misma ruta de directorios, no puede haber dos clases con el mismo nombre en el mismo package. Pero puede haber clases en diferentes packages, que tengan el mismo nombre.

Físicamente, un package no es otra cosa que una carpeta o directorio, que tiene un nombre que lo identifica y una ruta física para llegar hasta él desde una carpeta o directorio origen. El nombre completo de un package se forma escribiendo el nombre de todas las carpetas desde la carpeta origen hasta la carpeta del package, separando esos nombres con un punto. Así, cuando hablamos del package java.awt (nos pusimos nostálgicos, Abstract Windowing Toolkit - antiguo paquete Java de clases para implementación de ventanas), estamos diciendo que existe una carpeta java, la cual a su vez contiene una carpeta awt, y en esa carpeta se guardan las clases que conocemos para la gestión de interfaces del AWT:



Tenga en cuenta que un package finalmente es la forma en que se agrupan las clases frente al intérprete o máquina virtual Java (JVM), y que desde esta perspectiva finalmente lo que un package contiene son los archivos compilados de las clases (o sea, los archivos .class) Por razones prácticas obvias, se distribuyen en paquetes también los archivos fuente de un proyecto, pero estrictamente hablando un package es la forma en que se organizan los archivos .class de un proyecto para favorecer el trabajo de la JVM a la hora de encontrar, cargar e interpretar esos archivos para proceder a su ejecución.

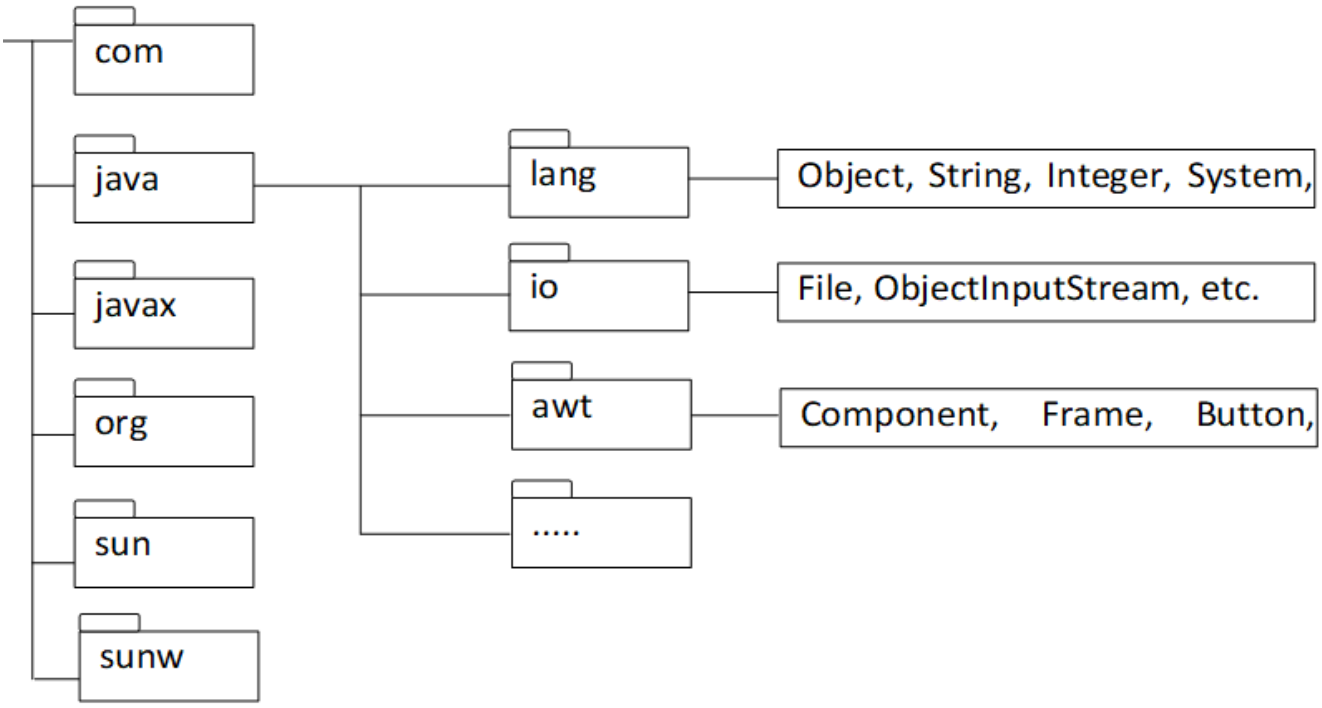
Clases nativas de Java - El uso de `import`

Todas las clases nativas de Java vienen en packages predefinidos, y todos esos packages a su vez vienen comprimidos y distribuidos en un archivo llamado .jar que forma parte de la librería de Java. Cuando se ejecuta un programa, la JVM busca dentro de rt.jar los archivos compilados de las clases nativas que se estén usando en ese programa, y desde allí los extrae, los carga, y los ejecuta. Dentro de ese archivo, están contenidas las carpetas origen de todos los packages que hemos estado usando hasta aquí, que son, entre otras, las carpetas java y javax. Prácticamente todos los paquetes y clases de la distribución estándar de Java

(o JSE) vienen en la carpeta java de rt.jar. Y también sabemos que la carpeta javax contiene al package swing que incluye casi todas las clases de Swing para el desarrollo de interfaces avanzadas de usuario.

El gráfico de la página siguiente (ver Figura 1) muestra un esquema del contenido de rt.jar. Recuerde que ese archivo contiene los archivos de código de byte o .class de las clases nativas de Java. No obstante, también existe un archivo src.zip en el directorio donde se instala el JDK (que suele ser c:\Program Files\java\open-jdk-17 o similar) que contiene la misma estructura de paquetes ya vista en rt.jar, pero ahora conteniendo los archivos fuente (archivos .java) de todas las clases Java.

Para que una clase pueda acceder a otras clases que se encuentran en un package distinto, se usa la instrucción import al comienzo del archivo de la clase. Con esa instrucción, lo que se está haciendo es indicarle a la JVM en qué lugar se encuentran las clases que se están usando. Por ejemplo, si una clase necesita acceder a diversas clases del AWT, podría hacerlo así: import java.awt.*; public class Ejemplo { private Button b1, b2; private Label l1, l2, …. }



De esta forma, la JVM sabrá dónde ubicar el código compilado de las clases Button y Label y de cualquier otra clase que esté en java.awt. Otra manera (más clara y específica, pero más larga...) consiste en usar el nombre de la clase completo dentro de la instrucción import, y repetir esta técnica con cada clase que se desee usar:

```
import java.awt.Button;
import java.awt.Label;

public class Ejemplo
{
    private Button b1, b2;
    private Label l1, l2,
    ….
}
```

Cuando una clase se nombra anteponiendo el nombre completo del package al nombre de la clase, se dice que el nombre de la clase está completamente calificado. Se puede prescindir de la instrucción import, si se está dispuesto a calificar completamente el nombre de cada clase que vaya a usar en cada lugar donde la misma se use:

```
public class Ejemplo
{
    private java.awt.Button b1, b2;
    private java.awt.Label l1, l2,

    public Ejemplo()
    {
        b1 = new java.awt.Button("Ok");
        l1 =  new java.awt.Label("Ejemplo");
    }
    ….
}
```

Esto último será necesario si en un mismo programa se usan dos clases con el mismo nombre pero provenientes de packages diferentes: la única forma de distinguirlas, será calificando completamente el nombre de ambas...

Notemos que el único package que no necesita una instrucción import explícita para poder acceder a sus clases, es el package java.lang que contiene las clases básicas del lenguaje (como Object, String, StringBuffer, Integer, Float, System, Math, etc.) El package java.lang es importado automáticamente al cargar la clase, de modo la JVM siempre sabrá dónde buscar las clases básicas de Java. Todo otro package, requerirá la instrucción import o la calificación completa del nombre de sus clases.

Por otra parte, un programador puede dividir las clases de sus proyectos en packages propios, en forma simple. Para empezar, al comienzo del archivo fuente de la clase se debe incluir la instrucción package, seguida del nombre del paquete (o carpeta) al cual pertenecerá la clase. Sólo puede haber una instrucción package en una clase, y debe estar al comienzo del archivo fuente. Si luego necesita usar clases de otros packages, use la instrucción import tantas veces como requiera, pero debajo de la instrucción package. El siguiente código muestra la forma de indicar que la clase Ejemplo pertenece al package prueba (y al no indicarse la ruta de carpetas para llegar a prueba, se entenderá que esa carpeta está en el mismo directorio que el proyecto):

```
package utnfc.isi.back.ejemplo;

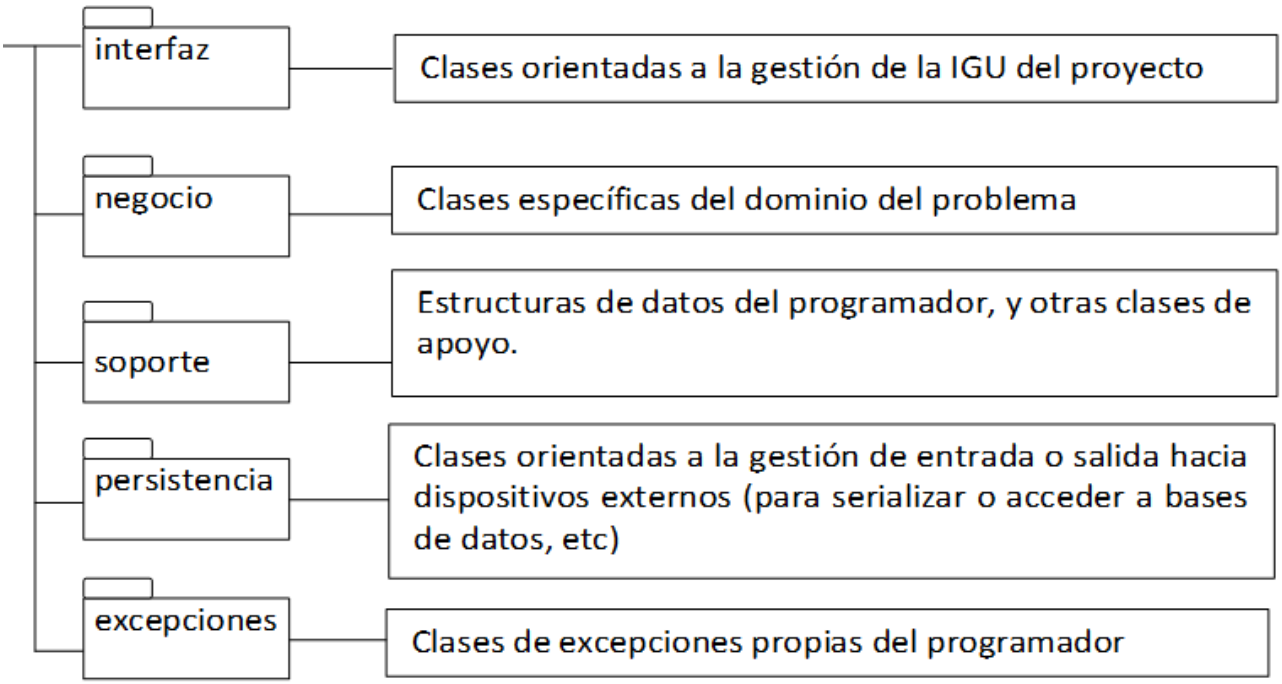
import java.awt.*;
import java.io.*;

public class Ejemplo
{
    private Button b1, b2;
    private Label l1, l2,
    ….
```

```
}
```

Por supuesto, deberá crear las carpetas que correspondan a cada package que quiera disponer en su proyecto. Eso puede hacerlo mediante el sistema operativo, o puede confiar en el IDE de desarrollo que esté usando para programar (BlueJ, NetBeans, Eclipse, etc.) Todos ellos pueden crear las carpetas de cada proyecto, en forma muy sencilla, y luego el programador puede agregar clases a uno u otro package. El propio IDE se encargará de colocar la instrucción package al inicio del archivo de la clase.

En general, el programador es dueño de decidir qué packages crear, con qué nombres designarlos y qué clases agrupar en ellos. Sólo debe tener en cuenta una pequeña convención para el nombre de un package: ese nombre debería designarse comenzando con una letra minúscula. En general, a partir de la versión 1.4 de Java no se espera que una clase esté fuera de todo package (o sea,, toda clase debería explícitamente ubicarse dentro de un package). Si no se usa la instrucción package para una clase, se asume que la misma pertenece al "package default" (que no es otra cosa que el directorio del proyecto que se está desarrollando), pero como dijimos, debería evitar esa situación. Una distribución de paquetes muy útil, aunque cada programador puede diseñar su propia distribución (y según los patrones de diseño que aplique tendrá eventualmente varias combinaciones), es la siguiente:



Empaquetado y distribución de artefactos de software

Cuando se termina de desarrollar una aplicación o un paquete de clases para ser usado por otros programadores, es buena idea pensar en comprimir y empaquetar esa aplicación, para luego distribuir el archivo empaquetado con más sencillez y poder usarlo como un todo en otras aplicaciones. Piense en lo que hace Java con el ya citado archivo rt.jar: prácticamente todas las clases nativas del lenguaje vienen distribuidas y organizadas adecuadamente en ese archivo, el cual se usa simplemente como fuente de una gran librería de clases.

En el directorio bin del JDK existe un programa, llamado jar (que sería jar.exe en la plataforma Windows) cuyo objetivo es producir un archivo comprimido y empaquetado con todo el contenido de un proyecto. La aplicación jar.exe toma la carpeta que se le indique, la cual debe contener un proyecto Java ya compilado, y crea un único archivo de salida con extensión .jar (que es abreviatura de Java Archive). Esencialmente, un archivo con extensión .jar no es otra cosa que un archivo comprimido con formato .zip, y por lo tanto puede ser luego abierto con cualquier compresor que reconozca ese formato (WinRar por ejemplo). Un detalle adicional a considerar, es que un archivo .jar puede contener una aplicación Java lista para ejecutarse (o sea, una aplicación que contiene un método main() en alguna de sus clases) o puede tratarse de una simple librería de clases distribuidas entre uno o mas packages pero sin método main() disponible. Si el .jar contiene un main(), el archivo .jar se conoce como un "jar ejecutable" y su principal característica es que el programa contenido en él, podrá ejecutarse directamente haciendo doble click sobre él desde el sistema operativo (por cierto, lo que realmente ocurrirá al hacer ese doble click, es que el sistema operativo transferirá la responsabilidad de ejecutar el contenido del archivo a la JVM que esté instalada en ese computador...).

Se puede crear un archivo .jar desde la línea de órdenes del sistema operativo, llamando directamente a jar.exe y pasándole los parámetros y modificadores que requiera, o se puede crear un .jar desde dentro de un IDE de desarrollo: la mayor parte de estos IDE permiten crear archivos .jar con mucha sencillez, ocultando los detalles de creación al desarrollador. Si está trabajando con BlueJ, por ejemplo, asegúrese de tener compilado el proyecto, luego seleccione la opción "Project" del menú principal, y dentro del menú que aparezca seleccione a su vez la opción "Create Jar File". En la ventana de diálogo que sigue, deberá ajustar algunas opciones de configuración (como el nombre de la clase que contiene al main() si es que hay alguna, o indicar si desea incluir los fuentes en el archivo a crear) y luego presionar Ok. A continuación se le pedirá el nombre del archivo .jar que quiere crear y la carpeta donde debe crearse, y finalmente el archivo .jar se creará en esa carpeta.

Si está trabajando con NetBeans, para crear el archivo .jar de su proyecto deberá hacer click derecho con el mouse al proyecto. Luego seleccionar la opción "Properties" en el menú emergente. En la ventana de configuración que aparece, seleccione el ítem "Build" y dentro de él elija el ítem "Packaging". Verá a la derecha de la ventana unas pocas opciones de configuración y algunos elementos que le indican donde será ubicado el archivo y cómo se llamará (en este caso, note que el archivo será colocado en el directorio dist, que será creado dentro del directorio del proyecto, y que el archivo se llamará igual que el proyecto pero con extensión .jar) Entre las pocas opciones de configuración que vea, simplemente asegúrese de tener tildada la casilla que dice "Build Jar after compiling". Eso es todo. De aquí en adelante, cada vez que el proyecto se compile, será creado el archivo .jar y será guardado en el directorio dist dentro del mismo proyecto. Si ya existía el archivo, será regenerado. Simplemente, asegúrese que una vez que haya terminado de desarrollar su aplicación haga una última "gran compilación" para que se genere el .jar definitivo y correcto.

Si damos un pequeño vistazo al contenido de un archivo .jar, notaremos que el mismo empaqueta la misma estructura de carpetas en la cual se distribuyeron las clases originales y dentro de esas carpetas aparecen los archivos .class (podrían aparecer también los .java si no los excluyó al crear el archivo .jar) Note que figura una carpeta llamada META-INF y que dentro de ella hay un archivo MANIFEST.MF. Ese archivo es un simple archivo de texto, que contiene unas pocas líneas que indican a la JVM algunos elementos para interpretar el contenido del .jar que se está abriendo. Sólo es interesante remarcar que si el archivo es un jar ejecutable, entonces el archivo MANIFEST.MF contendrá una línea comenzando con la expresión Main-Class: en la cual estará indicado el nombre de la clase del proyecto que contiene al método main() que debe usarse para ejecutar el programa. Si esa línea no viene acompañada de ningún nombre de clase, entonces el .jar no es un

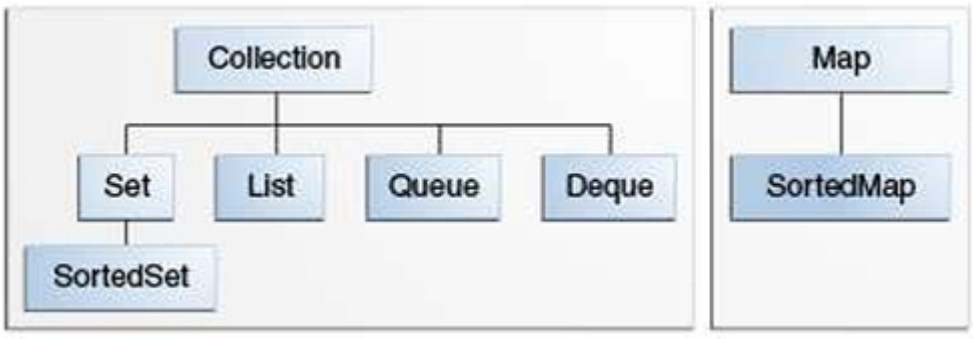
ejecutable y al hacer doble click sobre él probablemente se ejecute la máquina virtual pero provoque un error indicando que no existe un main class en el archivo MANIFEST.mf.

Api de colecciones

El lenguaje Java dispone de un complejo esquema de jerarquías de clases e interfaces nativas ya previstas para llevar adelante esta estrategia, en el marco de trabajo definido por el package java.util que ya hemos citado.

En general, para referirse a cada estructura de datos provista por ese paquete, se usa el nombre Colección (o Collection). Los objetos Collection representan estructuras de datos capaces de contener objetos (y no tipos primitivos). En ese sentido, un objeto Collection difiere de los arreglos comunes en cuanto a que estos últimos pueden contener objetos, pero también tipos primitivos. Sin embargo, un arreglo común tiene tamaño fijo, en tanto que un objeto Collection puede crecer a medida que lo requiere el programa. Por otra parte, si se desea almacenar valores de tipos primitivos en un objeto Collection, se pueden usar las clases de envoltorio (wrapper classes: Integer, Character, Double, etc.) sin problemas (y a partir de las últimas versiones de Java está también disponible el autoboxing, que facilita el trabajo del programador: si se desea almacenar valores de tipos primitivos en una colección, simplemente se invoca a los métodos de esa colección para hacerlo y el mecanismo de autoboxing de encarga de convertir esos valores primitivos a objetos wrapper).

El paquete java.util provee una jerarquía amplia y completa de interfaces y clases que representan a cada estructura de datos básica. Existen dos clases de interface que dan origen a sendas jerarquías para representar estructuras de datos: Collection y Map. La novedad es que estas jerarquías comienzan con una jerarquía de interfaces: desde Collection y Map se definen nuevas interfaces que derivan de Collection o Map, y a partir de ellas vienen las clases para cada estructura. Como vemos, se comienza con clases de interfaces que heredan entre ellas. Un esquema de las dos jerarquías de interfaces es el siguiente1:



Mostramos aquí un breve resumen del fundamento de cada una de ellas:

- Collection: es la raíz de una de las jerarquías de colecciones. Una colección representa un grupo de objetos que se conocen como sus elementos. La interface Collection es el denominador común que muchas de las colecciones implementan y se usa cuando se desea el máximo grado de generalidad en aplicaciones polimórficas. Algunos tipos de colecciones permiten elementos repetidos y otros no. Algunas colecciones son ordenadas y otras no. La plataforma Java no provee clases concretas que implementen directamente a Collection, pero sí provee varias que implementan a sus sub-interfaces específicas Set, List, Queue y Deque.
- Set: una colección que no contiene elementos repetidos. Esta interface modela la abstracción matemática del conjunto y se usa para representar conjuntos de diversos tipos de objetos.

- List: una colección o secuencia ordenada (en el sentido de preservar del orden de inserción). Puede contener elementos repetidos. Normalmente el programador que usa un objeto List tiene control preciso de en qué lugar de la lista se insertan los elementos y puede acceder a ellos mediante sus índices o posiciones.
- Queue: una colección usada para soportar y almacenar múltiples elementos antes de su procesamiento. Una Queue típicamente (pero no necesariamente) mantiene sus elementos en orden FIFO (first-in, first-out). No obstante, puede mantenerse el orden de acuerdo a algún criterio y obtener una cola de prioridad. Sea cual sea el orden de almacenamiento usado, el primer elemento de una Queue es el elemento que debería removido al invocar al método remove() o al método poll(). En una Queue de tipo FIFO, todo nuevo elemento se inserta al final de la estructura. Otros tipos de Queues deben especificar sus propias reglas.
- Deque: una colección usada para soportar y almacenar múltiples elementos antes de su procesamiento. Una Deque puede ser usada tanto en forma FIFO (first-in, first-out) como en forma LIFO (last-in, first-out). En una deque todo nuevo elemento puede ser insertado, recuperado y eliminado desde cualquiera de sus dos extremos.
- Map: un objeto que puede mapear ciertas claves hacia ciertos valores. Un Map no puede contener claves duplicadas; y cada clave puede mapear a lo sumo a un valor. Las tablas hash son posiblemente las formas más conocidas de implementación de un Map.
- SortedSet: un Set que mantiene sus elementos en orden ascendente. Es simplemente la versión ordenada de la interface Set. Provee algunas operaciones adicionales para aprovechar la ventaja de tener sus elementos ordenados.
- SortedMap: un Map que mantiene los elementos mapeados en él en orden ascendente de sus claves. Es simplemente la versión ordenada de la interface Map.

Estas jerarquías de interfaces brindan el contexto general y polimórfico básico para la implementación de las estructuras de datos nativas ya provistas por Java, y para el diseño de nuevas estructuras abstractas que pudiera necesitar el programador.

Para aprovechar al máximo este contexto de trabajo, es bueno hacer un paréntesis y poner en claro algunas ideas referidas a las estructuras de datos que por el momento nos ocupan: las listas. Estrictamente hablando, una lista es una estructura de datos lineal (cada elemento tiene o puede tener un único elemento sucesor y un único elemento antecesor) que además es adaptativa: a medida que necesita (o deja de necesitar) memoria, puede crecer o decrecer en su tamaño, y de tal forma además que un nuevo elemento puede ser insertado en cualquier lugar de la lista (y no necesariamente al principio o al final). Esta capacidad adaptativa es la que distingue a las listas de los arreglos. En muchos de los primeros lenguajes (o incluso en versiones antiguas de lenguajes modernos) los arreglos se definían como colecciones lineales esencialmente no adaptativas (no podían crecer ni decrecer en tamaño) pero que tenían la ventaja del acceso directo a sus componentes en tiempo constante. Esto se debe a que un arreglo se aloja completo en un bloque contiguo de memoria, y conocida la dirección del primer elemento es simple y directo calcular la dirección de cualquier otro.

Con el tiempo, y a partir de la flexibilidad para el manejo de memoria de lenguajes como C o C++ (en los que se programan muchos de los lenguajes modernos…) todos los lenguajes comenzaron a brindar capacidad adaptativa a sus arreglos (ya sea en forma directa o facilitando funciones de librería o clases especiales para hacerlo), llegando a un estado en el que la línea que separa a los arreglos de las listas se ha vuelto difusa: baste recordar (por ejemplo) que en el lenguaje Python directamente no se habla ya de arreglos sino de listas, y que esas listas en Python son en todo equivalentes a las java.util.ArrayList de Java, o a nuestras ArrayList,

estructuras lineales adaptativas, que se alojan en un bloque contiguo de memoria y disponen por tanto de acceso directo en tiempo constante.

De acuerdo a la forma en que una lista se implemente, se podrá contar con ciertas ventajas y desventajas. Esencialmente, hay dos formas básicas de implementar una lista:

- Implementación encadenada (o ligada): Cada elemento de la lista se almacena en un objeto separado designado como nodo. Cada nodo contiene un campo o slot para almacenar al elemento que contiene, y también incluye uno o dos campos más para almacenar las direcciones de su nodo sucesor y de su nodo antecesor. Si sólo se almacena la dirección de uno de ellos, la lista se dice simplemente vinculada, y si se almacenan los dos, se dice doblemente vinculada. La dirección del primero de los nodos se almacena en una variable separada (y también la del último si la lista es doblemente vinculada). Cuando la lista necesita crecer, se crea un nuevo nodo y se engancha al mismo en el lugar requerido, cambiando los valores de los enlaces al sucesor y/o antecesor. Si la lista necesita decrecer, se sueltan los enlaces que apuntan al nodo que se desea eliminar y se los ajusta para que apunten al nuevo sucesor y/o antecesor. Puede verse que de esta forma, la memoria ocupada por la lista no constituye un bloque contiguo (cada nodo se crea por separado y se aloja en el lugar que esté libre en ese momento) y esto lleva a que el acceso a cada nodo sólo pueda hacerse mediante un recorrido secuencial comenzando desde el principio (o desde el final) y siguiendo los enlaces al sucesor o antecesor. En otras palabras: el acceso a un elemento individual de una lista ligada requiere tiempo lineal (O(n)) en el peor caso. Un ejemplo de este tipo de implementación es la clase java.util.LinkedList de Java (doblemente vinculada).
- Implementación sobre un arreglo de soporte: Toda la lista se almacena sobre un arreglo de forma de ir ocupando ese arreglo de izquierda a derecha, sin saltear casillas. El arreglo de soporte puede contener más casillas que las que realmente está usando la lista en un momento dado, pero como la memoria ocupada es contigua se puede acceder en forma directa y en tiempo constante (O(1)) a cada elemento a través del índice que el mismo tiene en el arreglo. Si la lista tiene que crecer y el arreglo de soporte no tiene lugar libre, se crea un nuevo arreglo más grande, se copian los elementos del viejo arreglo al nuevo y se desecha el viejo, y se procede a la inversa cuando el arreglo debe acortarse. Obviamente, los dos ejemplos más directos que podemos citar son la clase java.util.ArrayList de Java, y nuestra ArrayList.

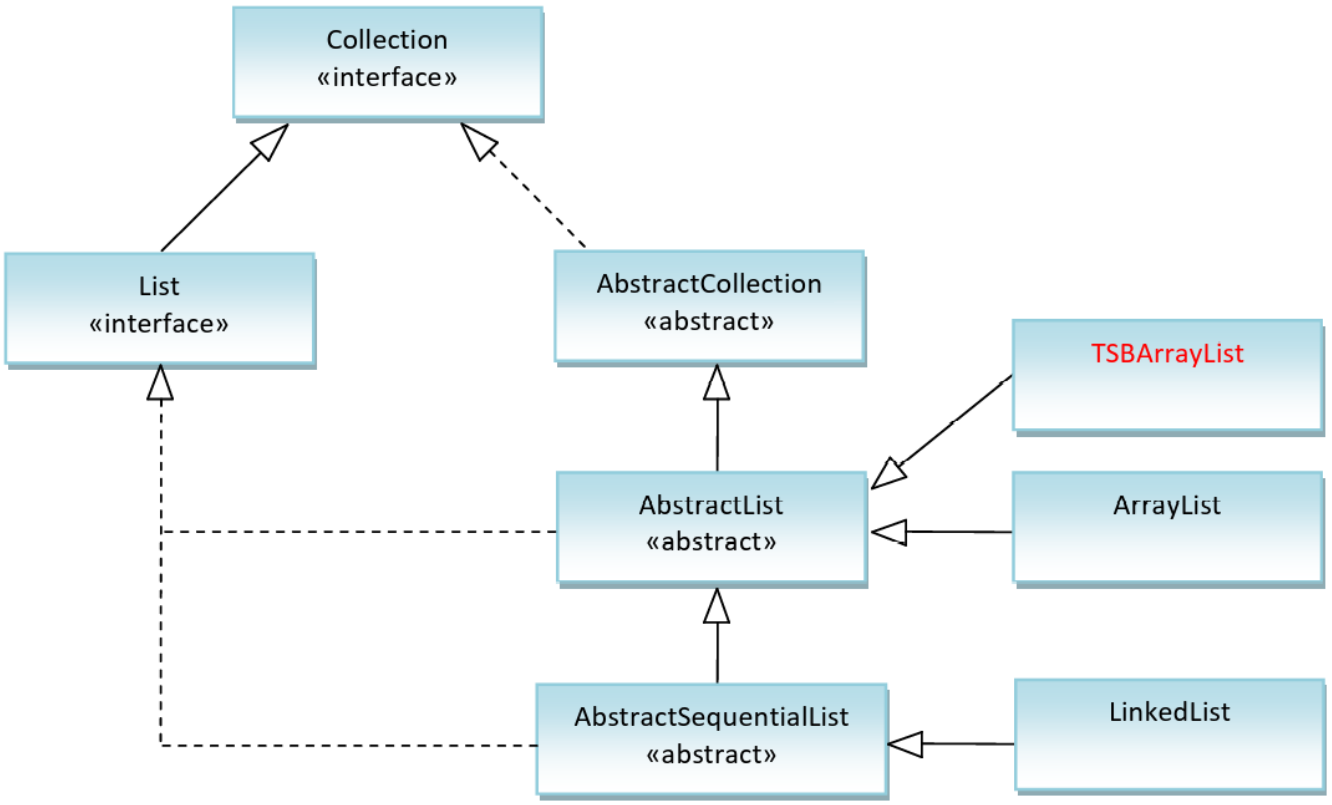
Interface List<E> extends Collection<E>

Tipo de salida	Método – Descripción básica
boolean	add(E e) Agrega el elemento especificado al final de esta lista (operación opcional).
void	add(int index, E element) Inserta el elemento especificado en la posición especificada en esta lista. (operación opcional).
boolean	addAll(Collection<? extends E> c) Agrega todos los elementos de la colección especificada, al final de esta lista, el orden en que son retornados por el iterador de la colección (operación opcional).
boolean	addAll(int index, Collection<? extends E> c) Inserta en esta lista todos los elementos de la colección especificada, a partir de la posición especificada (operación opcional).
void	clear() Elimina todo el contenido de esta lista (operación opcional).
boolean	contains(Object o) Retorna true si esta lista contiene al elemento especificado.
boolean	containsAll(Collection<?> c) Retorna true si esta lista contiene a todos los elementoss de la colección especificada.
boolean	equals(Object o) Compara el objeto especificado, con esta lista, para determinar si son iguales.
E	get(int index) Retorna el elemento en la posición especificada de esta lista.
int	hashCode()

	Retorna un valor de hash code para esta lista.
int	indexOf(Object o) Retorna el índice de la primera ocurrencia del elemento especificado en esta lista, o el valor -1 si la lista no contiene a ese elemento.
boolean	isEmpty() Retorna true si la lista no contiene elementos.
Iterator<E>	iterator() Retorna un iterador sobre los elementos de esta lista, en una secuencia apropiada.
int	lastIndexOf(Object o) Retorna el índice de la última ocurrencia del elemento especificado en esta lista, o el valor -1 si la lista no contiene a ese elemento.
ListIterator<E>	listIterator() Retorna un iterador de lista sobre los elementos de esta lista, en una secuencia apropiada.
ListIterator<E>	listIterator(int index) Retorna un iterador de lista sobre los elementos de esta lista, en una secuencia apropiada, comenzando desde la posición especificada.
E	remove(int index) Elimina el elemento ubicado en la posición especificada en esta lista (operación opcional).
boolean	remove(Object o) Elimina la primera ocurrencia del elemento especificado en esta lista, si está presente (operación opcional).
boolean	removeAll(Collection<?> c) Elimina de esta lista todos los elementos que están contenidos en la colección especificada (operación opcional).
default void	replaceAll(UnaryOperator<E> operator) Reemplaza cada elemento de esta lista con el resyultado de aplicar el operador especificado en ese elemento.
boolean	retainAll(Collection<?> c) Retiene en esta lista sólo los elementos que están contenidos en la colección especificada (operación opcional).
E	set(int index, E element) Reemplaza el elemento ubicado en la posición especificada, con el elemento especificado (operación opcional).
int	size() Retorna el número de elementos de esta lista.
default void	sort(Comparator<? super E> c) Ordena esta lista de acuerdo al orden inducido por el Comparator especificado.
default Spliterator<E>	spliterator() Crea un Spliterator sobre los elementos de esta lista.
List<E>	subList(int fromIndex, int toIndex) Retorna una vista de una porción de esta lista entre el índice *fromIndex*, inclusive, y

	el índice toIndex, exclusive.
Object[]	toArray() Retorna un arreglo conteniendo todos los elementos de esta lista en una secuencia apropiada (desde el primero hasta el úlitmo).
<T> T[]	toArray(T[] a) Retorna un arreglo conteniendo todos los elementos de esta lista en una secuencia apropiada (desde el primero hasta el úlitmo); el tipo de tiempo de ejecución del arreglo retornado es el del arreglo especificado.

Para ahorrar algo de trabajo al programador, Java provee ciertas clases abstractas que ya implementan la interface List y proveen una implementación adecuada de la mayor parte de estos métodos (y de otros que a su vez provienen de la interface Collection). Dos de esas clases son muy útiles: AbstractList y AbstractSequentialList (ver diagrama de clases siguiente):



La clase abstracta AbstractCollection provee una implementación esencial de los métodos de la interface Collection, y de ella deriva a su vez la clase AbstractList que es super clase de la clase AbstractSequentialList. Y las dos últimas implementan la interface List. La clase abstracta AbstractList provee una implementación básica (a modo de plantilla) de la interface List para minimizar el esfuerzo requerido para implementar esa interface cuando se diseñan estructuras lineales basadas en almacenamiento de acceso directo (como un arreglo, o una lista soportada sobre un arreglo). Para la implementación de estructuras lineales basadas en acceso secuencial (como una lista ligada), debería usarse como base la clase AbstractSequentialList en lugar de AbstractList. De hecho, en la implementación nativa de Java para su clase java.util.ArrayList, la misma deriva de AbstractList, mientras que java.util.LinkedList deriva de AbstractSequentialList (ver diagrama de clases anterior).

Por lo tanto, si un programador desea implementar alguna clase que permita modelar una lista basada en un soporte de acceso directo, puede emular lo que hace Java con su propia clase java.util.ArrayList, derivarla de la clase AbstractList, la cual le facilita mucho trabajo ya desarrollado. Y si desea implementar una clase para

modelar una lista basada en la idea de lista ligada, debería pues derivarla desde AbstractSequentialList y aprovechar lo que esa clase ya provee. En nuestro caso, la clase ArrayList se ha hecho derivar desde AbstractList. Lo queda es analizar las convenciones y recomendaciones que la propia documentación provee para cada una de estas clases cuando se hace un trabajo similar al que vamos a intentar. Veremos que en esencia no es demasiado trabajo:

Si se deriva desde AbstractList (para una lista con soporte de acceso directo):

1. Si la lista es inmodificable, el programador sólo necesita derivar esta clase y proveer implementaciones para los métodos get(int) y size().
2. Si la lista es modificable (esto es, algún elemento pre-existente puede cambiar por otro, pero sin cambiar el tamaño de la lista), el programador debe adicionalmente sobre-escribir el método set(int, E) (el cual en caso contrario lanzará una excepción de la clase UnsupportedOperationException al ser invocado).
3. Si además el tamaño de la lista puede cambiar (ya sea que aumente o disminuya), entonces el programador debe adicionalmente sobre-escribir los métodos add(int, E) y remove(int).
4. El programador debería también proveer una implementación del constructor sin parámetros, y también del "constructor collection" (es decir, del constructor que toma como parámetro a otra colección y crea la lista copiando el contenido de esa otra colección).
5. A diferencia de otras implementaciones de colecciones abstractas, en este caso el programador no está obligado a implementar una clase iteradora ni tampoco a dar una implementación de los métodos iterator() y listIterator(): la clase AbstractList ya provee una implementación apropiada basada en el uso de los métodos de acceso directo get(int), set(int, E), add(int, E) y remove(int).
6. Finalmente un detalle técnico: la clase AbstractList dispone de un atributo protegido de tipo int, llamado modCount. Ese atributo se usa para llevar la cuenta de la cantidad operaciones realizadas sobre la lista que modificaron su estructura, es decir, operaciones que cambiaron el tamaño de la lista. Usando ese atributo, las clases iteradoras pueden inferir si la lista ha cambiado mientras se está realizando concurrentemente un recorrido con algún iterador, y en ese caso, interrumpir de inmediato el proceso iterador lanzando una excepción de la clase ConcurrentModificationException. Si esto se aplica, se dice que la clase sigue una estrategia de fail-fast iterator (o iterador de respuesta rápida a fallos). No hay que alarmarse por este detalle: si el programador desea una implementación fail-fast iterator, lo único que debe hacer es sumar 1 al atributo modCount en cada método que efectivamente lleve a cabo una modificación estructural de la lista (operación que implique un cambio de tamaño). El control de la falla en un contexto concurrente y el lanzamiento de la excepción, es llevado a cabo por los métodos ya implementados en la clase AbstractList. Y si el programador no desea una implementación fail-fast iterator, simplemente puede ignorar el atributo modCount y dejar que cada iterador haga lo que pueda cuando recorra una lista y esta pueda cambiar durante el recorrido (en este caso, no hay seguridad de obtener resultados correctos, e incluso podría ocurrir de todos modos una excepción si el iterador llega a un punto (por ejemplo) en el que no pueda recuperar el próximo elemento).

Si se deriva desde AbstractSequentialList (para una lista ligada):

1. Para implementar una lista ligada, el programador sólo necesita derivar esta clase y proveer implementaciones para los métodos listIterator() y size() (y obviamente deberá proveer una clase iteradora que implemente ListIterator). Si la lista es inmodificable, el programador sólo necesita implementar los métodos hasNext(), next(), hasPrevious(), previous() e index() de la clase iteradora que programe.

2. Si la lista es modificable (esto es, algún elemento pre-existente puede ser reemplazado por otro pero sin cambiar el tamaño de la lista) entonces el programador debería adicionalmente implementar el método set() de la clase iteradora para listIterator().
3. Si la lista es de tamaño variable, entonces el programador debería adicionalmente implementar los métodos remove() y add() de la clase iteradora para listIterator().
4. El programador debería también proveer una implementación del constructor sin parámetros, y también del "constructor collection" (es decir, del constructor que toma como parámetro a otra colección y crea la lista copiando el contenido de esa otra colección).
5. También aquí es válido considerar las recomendaciones respecto al atributo modCount heredado desde AbstractList (ver para ello el punto f de las recomendaciones para implementar una lista derivando desde AbstractList.)

Tanto si se deriva desde AbstractList como si se lo hace desde AbstractSequentialList, la documentación para cada uno de los métodos no abstractos en cada clase describe su implementación en detalle. Y por supuesto, cada uno de esos métodos puede ser sobre escrito si la colección que está siendo diseñada admite implementaciones más eficientes.

Llegamos al fin a poder intentar nuestra propia colección en base al planteo inicial de ListaArray. Con estos elementos ya presentados, sólo nos queda ver en detalle la implementación final de nuestra clase ArrayList, que se define esencialmente así:

```
public class ArrayList<E> extends AbstractList<E>
                            implements List<E>, RandomAccess, Cloneable,
Serializable {
    ...

}
```

Note que los atributos de la clase son los mismos que ya hemos incluido en la versión original de la clase, con las mismas especificaciones y usos.

La interface RandomAccess es una interface vacía o de marcado, que sirve para que una clase notifique que admite operaciones de acceso directo a sus elementos (normalmente de tiempo constante). De esta forma, un algoritmo genérico (que tome un parámetro polimórfico) puede chequear si una instancia pertenece a una clase que haya implementado RandomAccess y modificar el comportamiento de tal algoritmo para aplicar técnicas más veloces que aprovechen el acceso directo. Por el momento, es suficiente con saber que el programador sólo debe indicar que la clase implementa RandomAccess. Y algo similar sucede con la interface Serializable (ya conocida) que también es una interface vacía cuya implementación se usa para notificar que la clase admite la serialización de sus instancias.

La interface Cloneable también es una interface vacía, pero en este caso hay algún trabajo extra para realizar: La implementación de esta interface se usa para notificar que la clase admite la operación de clonar sus instancias invocando al método clone() (que NO viene especificado en la interface Cloneable, sino en la clase Object, por lo que toda clase dispone de la implementación esencial del método). La idea es la siguiente: si una instancia de una clase X invoca a clone() pero la clase X no implementó Cloneable, se producirá una

excepción de la clase CloneNotSupportedException. La clase X debe implementar Cloneable si se tiene pensado invocar a clone() con sus instancias.

Si se usa directamente el método clone() heredado desde Object, este método creará y retornará una copia superficial del objeto: todos sus atributos de tipo primitivo serán efectivamente replicados y copiados en el nuevo objeto, pero para los atributos que sean referencias a otros objetos, este método replicará las referencias (sin copiar los objetos en sí mismos). Por lo tanto, posiblemente el programador querrá implementar su propia versión de clone() en las clases que implementen Cloneable. En ese caso, lo aconsejable es que en la nueva versión del método se comience invocando al método clone() de la super clase (y si todas las clases siguen esta convención, finalmente todas invocarán a Object.clone()). Con esto se garantiza que se creará un objeto de la clase correcta, con sus atributos primitivos correctamente replicados, quedando para el programador la tarea de modificar en la copia clonada los atributos que sean referencias. En nuestra clase ArrayList el método clone() se redefinió así:

```
public Object clone() throws CloneNotSupportedException {
    ArrayList<?> temp = (ArrayList<?>) super.clone();
    temp.items = new Object[count];
    System.arraycopy(this.items, 0, temp.items, 0, count);
    // fail-fast iterator para el nuevo objecto...
    // modCount se hereda desde AbstractList y es protected...
    temp.modCount = 0;
    return temp;
}
```

El método define en su cabecera que puede lanzar una excepción de la clase CloneNotSupportedException (lo cual teóricamente ocurrirá si se invocase a este método sin que la clase ArrayList implemente Cloneable... pero en la práctica eso no pasará, ya que efectivamente nuestra clase hizo esa implementación).

El método comienza creando el nuevo objeto (que finalmente será el que se retorne), pero invocando a super.clone() para hacer esa creación. Con esto, el nuevo objeto temp tendrá todos sus atributos con los mismos valores que el objeto actual (entre ellos, initial_capacity y count). Note que clone() retorna una referencia de la clase Object, y por eso se aplica una operación de casting explícito para convertir el tipo del resultado a ArrayList. Para evitar que la referencia temp.items apunte al mismo arreglo referenciado por this.item (que es el efecto de la copia superficial), se crea nuevamente y por separado el arreglo temp.items y se usa luego System.arraycopy() para copiar las direcciones contenidas en this.items (y en este momento, note que clone() replicará la lista, pero no los objetos contenidos en ella). Finalmente, se pone a cero el valor del atributo temp.modCount (heredado desde AbstractList) para activar desde cero el mecanismo fail—fast iterator en el objeto recién creado (y evitar que ese atributo tenga en temp el mismo valor que tenía en this...). Cuando todo esto ha sido concluido, el método termina retornando el nuevo objeto.

Como la clase ArrayLIst deriva de AbstractList, entonces la mayor parte del trabajo de implementar los métodos de la interface List está ya realizada (aunque sea a un nivel esencial). El contenido completo de la clase es el que puede consultarse en el ejemplo que acompaña el presente apunte.

El patrón Iterador en Java

Una revisión interesante a realizar es la implementación que tiene java del patrón iterador, dicha implementación se basa en las interfaces Iterable<> e Iterator<>. La interfaz Iterable obliga a la clase que la implementa a implementar el método:

```
    public Iterator<> iterator() {

    }
```

de forma tal que dicho método devuelva una instancia de iterador que será implementado en una clase interna, sin embargo, vale la pena observar que el uso de la interfaz Iterator, provoca un desacople entre el objeto iterador específico de la clase y quién requiera utilizarlo, es decir, quien requiera un iterador no necesita saber si está iterando un ArrayList de Backend o un ArrayList de Java o un LinkedList u otra clase Iterable, esto potencia y flexibiliza el modelo.

Sin embargo, para mostrar la implementación del Iterador como Java lo esperaría, hemos agregado también en el ejemplo que acompaña este apunte, entre las clases parametrizadas, la clase `ListaArrayIterable`, que implementa la interfaz `Iterable` y declara la clase interna `ListaArrayIterator` que implementa la interfaz `Iterator`.

En este caso la implementación cambia sutilmente respecto de la implementación de iterador que analizamos previamente, puesto que si bien mantiene el constructor como creador e iniciador del iterador como es de esperarse, y también el método `hasNext()` que determina si aún quedan elementos por recorrer. Agrega un solo método extra que cumple las funciones de `getActual()` y `siguiente()` y en este orden preciso, este método es el método `next()` que al mismo tiempo mueve el iterador al siguiente elemento y retorna el elemento actual.

Finalmente, el método remove() debe eliminar el último elemento que haya sido alcanzado por next() (es decir, el elemento que actualmente esté en la casilla current), pero de forma que current quede apuntando al elemento que estaba a la izquierda de este (es decir, current debe quedar valiendo el índice de la casilla anterior). Por lo tanto, primero debe chequear que next() haya sido invocado (lo cual hace con la condición de la primera línea), lanzando una excepción en caso contrario e interrumpiendo la ejecución del método. Si todo estaba bien con next(), entonces para llevar a cabo efectivamente el borrado simplemente se invoca al método remove(index) que ya está implementado en la propia clase ListaArrayIterable (y que ya hemos mostrado): al invocarlo, se le pasa como parámetro el valor current, y el método hace el resto, incluyendo la disminución del tamaño del arreglo si fuese necesario y la operación de restar uno al valor de count para reflejar el hecho de que la lista ha perdido un componente. Note que para invocar al método remove(index) de la clase ListaArrayIterable, el método remove() de la clase ListaArrayIterator accede al mismo de la forma siguiente:

```
        ListaArrayIterable.this.remove(current)
```

Esto es: si desde el interior de un método que pertenece a una clase interna (como el método remove() de la clase ListaArrayIterator) se quiere acceder a un método o atributo de su clase contenedora (como el método

remove(index) de la clase ListaArrayIterable) pero usando la referencia this, entonces el método de la clase interna debe anteponer el nombre de la clase contenedora antes de acceder a this (pues de otro modo, estará accediendo al puntero this de la propia clase interna...).

El método remove() finaliza volviendo a false el valor del atributo next_ok, la idea es que si remove() fue invocado es porque next() había sido invocado a su vez, pero una vez eliminado el elemento actual, next() debe volver a invocarse para poder activar nuevamente a remove(). Si remove() pone false en next_ok, entonces dos invocaciones seguidas a remove() (sin invocar a next() entre ambas) provocarán una excepción.

Anexo I: 🔚 Comparación de recorridos sobre colecciones

Enfoque evolutivo

Una vez construida una lista con objetos, es habitual necesitar recorrerla para inspeccionar, transformar o acumular datos. En esta sección final, ilustramos cómo ha evolucionado el enfoque de recorrido sobre estructuras como List, desde los métodos más acoplados con la estructura interna, hasta formas más genéricas y desacopladas que Java promueve a partir de sus interfaces Iterable e Iterator.

🧱 **1. Recorrido con subíndices (for tradicional)** Este enfoque solo es viable si la estructura interna de la lista soporta acceso aleatorio, como sucede en las listas sobre arrays (como ArrayList). Se accede a cada elemento por posición:

```
...
List<String> nombres = new ArrayList<>();
nombres.add("Ana");
nombres.add("Luis");
nombres.add("Carla");

for (int i = 0; i < nombres.size(); i++) {
    System.out.println(nombres.get(i));
}
...
```

🔍 En este caso:

- `size()` indica cuántos elementos tiene la lista.
- `get(i)` permite acceder a cada elemento directamente.
- Requiere que el almacenamiento subyacente sea indexado o permita el acceso por subíndice.

> Cabe mencionar que, más allá de la necesidad de conocer la estructura interna de la colección para realizar este recorrido, en el caso de los ArrayList que soportan acceso directo este tipo de acceso no provocan overhead, pero si estuviéramos hablando de una lista vinculada, este recorrido sería **Orden(n^2^)**

🔁 **2. Recorrido usando Iterator (while + patrón iterador)** Este enfoque sigue el patrón iterador tal como lo vimos en este apunte, y funciona sobre cualquier implementación de `Iterable`, sin importar cómo esté implementado internamente.

```
...
Iterator<String> it = nombres.iterator();
while (it.hasNext()) {
    String nombre = it.next();
    System.out.println(nombre);
}
...
```

🔍 En este caso:

- Se desacopla completamente del tipo de colección (ArrayList, LinkedList, etc.).
- Es útil incluso para estructuras sin acceso directo, ya que delega la responsabilidad de obtener el siguiente elemento a la estructura.
- Refuerza el patrón de diseño Iterator explicado en este material.
- Permite eliminar elementos durante el recorrido (`it.remove()`), con control de fail-fast.

🔁 **3. Recorrido con for-each (forma moderna y recomendada)** El `for-each` es azúcar sintáctico provisto por Java para recorrer cualquier colección que implemente `Iterable`. Bajo el capó utiliza un `Iterator`, pero sin necesidad de declararlo explícitamente.

```
...
for (String nombre : nombres) {
    System.out.println(nombre);
}
...
```

🔍 Características:

- Desacopla aún más al programador del mecanismo de iteración.
- Funciona con cualquier clase que implemente `Iterable<E>`.
- Es la forma recomendada para recorrer colecciones en la mayoría de los casos simples.
- No permite eliminar elementos durante el recorrido (no expone `remove()` del Iterator).

☑ Conclusión

A medida que avanzamos en abstracción, pasamos de depender del almacenamiento interno (índices) a usar interfaces (`Iterable`, `Iterator`), y finalmente llegamos a formas sintéticas (for-each) que logran mayor claridad, flexibilidad y mantenibilidad.

Este recorrido gradual ilustra cómo la evolución del lenguaje Java acompaña los principios de diseño orientado a objetos: ocultamiento de implementación, uso de interfaces, y separación entre qué se hace y cómo se hace.